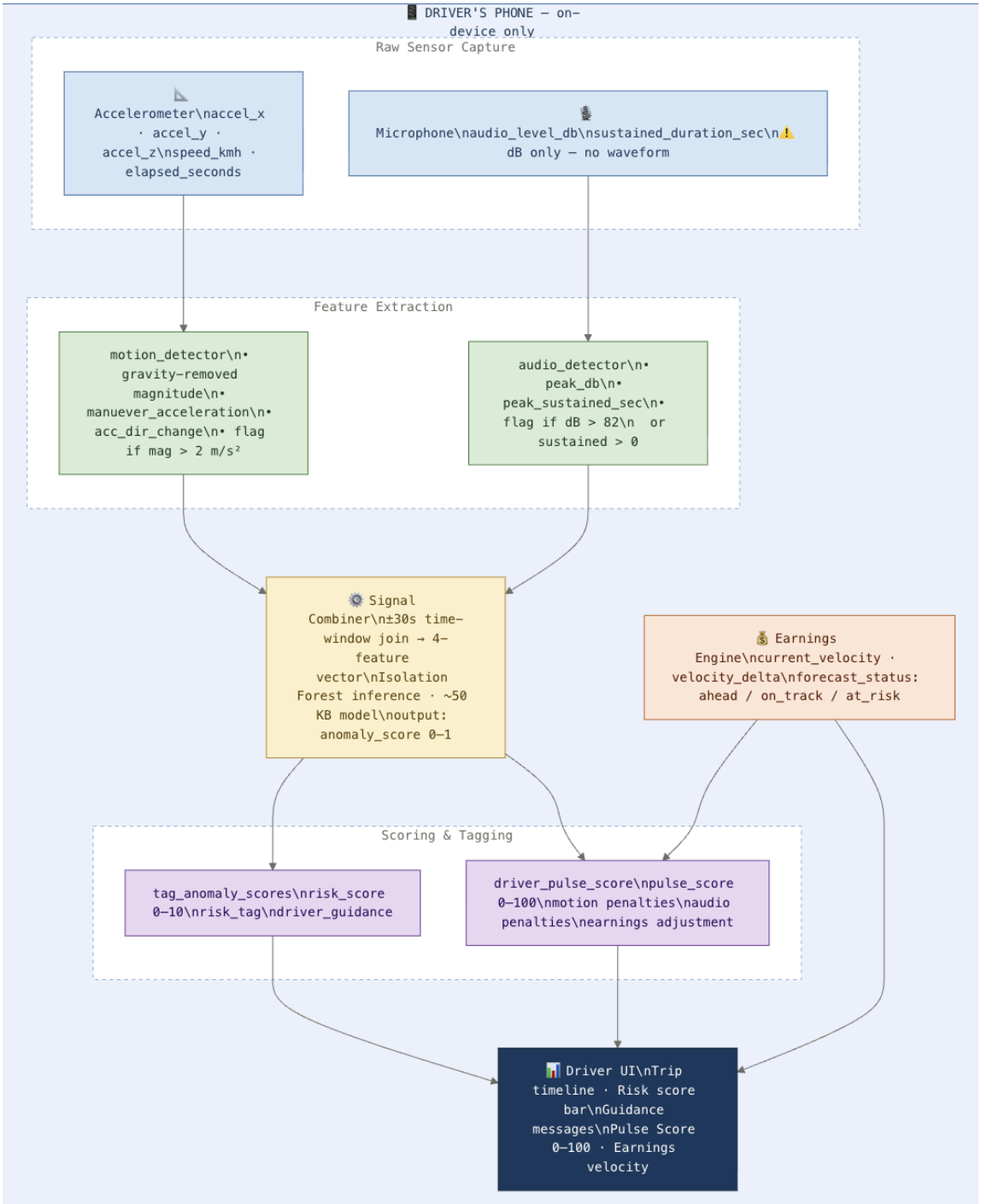
DRIVER PULSE

System Architecture

Team 5

1. Architecture Diagram



2. Data Flow

STEP 1: RAW SENSOR CAPTURE

Two sensor streams feed the pipeline:

- Accelerometer — accel_x, accel_y, accel_z, speed_kmh, elapsed_seconds
- Microphone — audio_level_db, sustained_duration_sec

Sampling interval: approximately 30 seconds (sparse by design, continuous sampling drains battery and produces more data than the model can use at this density).

The microphone never captures a waveform. The OS-level audio API is called only to return a dB reading for the current window which is equivalent to a sound level meter, not a recorder.

STEP 2: FEATURE EXTRACTION (MOTION_DETECTOR + AUDIO_DETECTOR)

Raw sensor values are noisy and uninterpretable on their own. Feature extraction converts them into four quantities that have physical meaning:

- **manuever_acceleration**
  - Computed as |Δ(gravity-removed magnitude)| between consecutive readings.
  - Gravity removal: magnitude = $\text{sqrt}(x^2 + y^2 + (z - 9.8)^2)$. This isolates lateral and longitudinal forces from gravitational baseline.
  - A bump on a flat road increases z but not the delta. Gravity removal ensures only genuine force changes are captured.
- **acc_dir_change**
  - Computed as the angular shift in the 3D acceleration vector between readings.
  - High acc_dir_change at low maneuver_acceleration = swaying, not a hard event.
  - High in both = genuine sharp maneuver.
- **peak_db**
  - Maximum dB reading recorded in the window. Captures event intensity.
- **peak_sustained_sec**
  - Total seconds the audio level stayed elevated. Captures event duration. A door slam has high peak_db but near-zero peak_sustained_sec. A passenger argument has high peak_db and high peak_sustained_sec.

STEP 3: TIME-WINDOW JOIN (SIGNAL_COMBINER)

Motion and audio are sampled on independent schedules and cannot be joined on exact timestamp. The +30/-30 second window is chosen to match the approximate sensor sampling interval. Readings within 30 seconds of each other on the same trip are treated as potentially co-occurring.

Readings with only one sensor (motion-only or audio-only) are retained. Missing features are imputed with the column median so the model receives a complete 4-feature vector. The signal_driver field records whether a window had motion only, audio only, or both.

STEP 4: ISOLATION FOREST INFERENCE

The saved model (isolation_forest.pkl, 50 KB) runs entirely on-device. No data is sent to a server to produce a score. Inference on a single window takes microseconds on any modern smartphone CPU.

The model was trained on 636 windows (40 trips) and evaluated on 589 windows (10 held-out trips, split by trip_id to prevent data leakage).

STEP 5: SCORING AND TAGGING

Two complementary outputs are produced:

- **tag_anomaly_scores**
  - The ML anomaly score (0-1) is scaled to 0-10 and mapped to a human-readable tier (smooth_trip → critical_moment) with an empathetic guidance message.
  - The message language is deliberately non-specific("elevated cabin noise" not "argument detected") because the system cannot verify what caused the signal.

- **driver_pulse_score**
  - A rule-based composite score starting at 100, deducting for detected events and adjusting for earnings pace.
  - Transparent, auditable, and trustworthy independent of model accuracy.

STEP 6: EARNINGS ENGINE

Runs independently of the sensor pipeline. Reads cumulative_earnings and elapsed_hours from the earnings log and computes velocity against the driver's personal target. The output (ahead / on_track / at_risk) feeds into both the Driver Pulse Score and the UI earnings panel.

3. Edge vs Cloud Processing

3.1 What runs on-device

Every component in the current pipeline runs on-device:

- Sensor reading (accelerometer + microphone dB)
- Feature extraction (motion_detector, audio_detector)
- Signal joining (signal_combiner)
- Isolation Forest inference (isolation_forest.pkl, ~50 KB)
- Scoring and tagging (tag_anomaly_scores, driver_pulse_score)
- Earnings computation (earnings_engine)

3.2 Why on-device for everything

- Latency– Drivers need feedback during or immediately after a trip. A round trip to a cloud server adds 200–800ms of latency. In a low-signal urban environment (tunnels, dense areas), cloud dependency means the feature may be unavailable precisely when the driver needs it.

- Cost— Streaming sensor data to a cloud inference endpoint for every active driver at 30-second intervals is expensive at scale. On-device inference with a 50 KB model has effectively zero marginal cost.

- Privacy— The strongest privacy guarantee that the data never leaves the device.

- Reliability— Drivers in dense urban areas frequently experience intermittent connectivity. A cloud-dependent pipeline degrades gracefully to nothing. An on-device pipeline works at full fidelity regardless of network state.

3.3 What would move to cloud in production

- Model retraining:
  - The Isolation Forest needs periodic retraining as real trip data accumulates. Retraining requires aggregated, anonymised feature vectors (4 numbers per window). Retrained weights are pushed back to the device as a small file update (50 KB).
- Population baselines:
  - A production system would maintain population-level distributions of manuever_acceleration and audio levels across all drivers in a city. Only aggregated statistics are needed: no individual trip data is required on the server.
- Earnings goals sync:
  - A driver's target_earnings and target_hours may be stored in their Uber account. Reading these once per shift is a single lightweight API call containing no sensor data.

3.4 What will never move to cloud

- Raw audio waveforms (never captured in the first place)
- Per-window sensor readings linked to a specific driver
- GPS coordinates at the trip level
- Any data that could reconstruct a driver's route or passenger interactions

4. Privacy and Resource Constraints

4.1 Audio privacy

Constraint: Passengers have a reasonable expectation that their conversations in a vehicle are private. Recording audio and transmitting it would require explicit consent, create legal liability, and undermine driver trust.

Design response:

- The microphone is used only to compute a dB level at 30-second intervals. This is qquivalent to reading a sound meter, not a recorder.
- sustained_duration_sec is the only audio-derived feature used in the model. It measures only how long noise stayed elevated, not what was said.

- audio_classification (argument, very_loud, etc.) is derived from dB thresholds, not speech recognition or audio content analysis.
- No audio data of any kind is transmitted off-device.

The system cannot by design:

- determine if a passenger said anything specific
- distinguish a passenger argument from loud music
- identify who was speaking

4.2 Location privacy

Constraint: GPS coordinates are present in the accelerometer dataset but were not used in any detection or scoring logic.

Design response:

Location data is explicitly excluded from the feature set. The system does not know where on a route an event occurred. A driver flagged for harsh braking could have been responding to a pedestrian, a pothole, or an emergency.

4.3 Behavioural profiling risk

Constraint: A system that scores driver behaviour continuously creates a de facto performance record. If that record feeds into platform decisions (deactivation, surge access, rating adjustments), it becomes a surveillance instrument.

Design response:

- Scores are displayed only to the driver, not to the platform.
- The guidance language is empathetic and non-punitive by design.
- The system explicitly cannot and does not distinguish driver fault from environmental factors (road quality, passenger behaviour).
- The product document does not propose feeding scores into any Uber platform metric.

4.4 Resource constraints

Battery:

- Accelerometer sampling at 30-second intervals draws negligible power. Most fitness trackers sample at 10x this frequency continuously.
- The microphone is active for a fraction of a second per 30-second window to capture a dB reading. It does not stay open between readings.
- Isolation Forest inference on a 4-feature vector takes around 1ms on a modern phone CPU. No GPU or neural inference engine required.

Storage:

- The model file (isolation_forest.pkl) is around 50 KB.
- Per-trip output is approx. 200 bytes per window. A 2-hour shift with readings every 30 seconds across 10 trips produces roughly 240 rows, which is under 50 KB of output data.
- Data older than 30 days can be safely purged with no impact on model inference (which is stateless per window).

Compute:

- All processing is CPU-bound and lightweight. No GPU required.
- The full pipeline (signal join → inference → scoring) runs in under 100ms for a single trip on a mid-range smartphone CPU.
- The Isolation Forest was chosen partly for inference efficiency. A neural network producing equivalent accuracy would require 10-100x more compute at inference time with no measurable benefit for tabular data at this density.

Network:

- Zero network usage during normal operation.
- Earnings goals sync: one lightweight API call at shift start.
- Model update: pushed as a 50 KB file on the same schedule as app updates.